

BLP v1.2 — data_test/ add-on: real-world continuous signing

Scope. Re-run of § 15.2 only, after two real Thai Sign Language videos were placed in data_test/ .

Everything else in [blp_v1.md](#) — architecture, training, closed-set and open-set results — is unchanged and was not retrained.

Run date: 2026-09-07 • conda `hugging` • RTX 2050 • notebook re-executed end to end in **16.4 min** (68 cells, 17 figures, 0 errors)

New code: [modules/continuous.py](#)

1 • What arrived, and why it broke the old assumption

file	resolution	fps	frames	duration
XXXXXXXXXXXXXXXXXXXXX.mp4	640×360	30	308	10.3 s
XXXXXXXXXXXXXXXXXXXXX.mp4	640×360	25	168	6.7 s

The filenames are **whole Thai clauses** — "*I comfort a crying friend*" and "*Shall we go eat together?*" — not gloss names. These are **continuous sentences**, each containing roughly four to five signs. The recogniser is trained on **isolated** signs with a median length of 54 frames (~1.8 s).

v1's data_test route fed the whole file to the classifier as one clip. That asks an isolated-sign model to name a sentence with a single word. It is the wrong question, and the model answered it the only sensible way:

video	prediction	p	open-set score	prototype cos
XXXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXX	—	0.093	−2.41	0.415

XXXXXXXXXXXXXXXXXXXX XXXXX	—	0.095	−2.52	0.396
-------------------------------	---	-------	-------	-------

(threshold −1.47; in-vocabulary test clips average prototype cos **0.767 ± 0.171**)

So the honest v1 answer was "unknown" — correct, but useless. That is what this add-on fixes.

2 • Before optimising: is the extraction even right?

No point tuning a decoder on top of a broken feature pipeline. Two checks were added (`CT.extraction_sanity` , `CT.distribution_shift`) and both are now cells in the notebook.

Landmark convention. In the corpus, slot 33 is the left-hand root and must track pose landmark 15 (left wrist); slot 54 must track pose 16. A handedness swap in the video extractor would mirror every downstream feature and be nearly invisible in the metrics.

source	slot33→L wrist	slot33→R wrist	slot54→R wrist	slot54→L wrist
TSL-ONE-S corpus (reference)	0.0289	0.1180	0.0182	0.0986
XXXXXXXXXXXXXXXXXXXX XXXXXXXXXX	0.0256	0.1356	0.0244	0.2319
XXXXXXXXXXXXXXXXXXXX XXXXXXXXXX	0.0327	0.2753	0.0219	0.2954

Each hand sits on its own wrist by a 4–13× margin, and `left_shoulder.x > right_shoulder.x`

holds for corpus and videos alike, so the mirroring convention agrees. **PASS.**

Detection rate. Pose 100 % of frames in both videos; hands 81 %/82 % and 95 %/92 % — at or above the corpus reference clip (78 %/71 %).

Feature-space distance. Mean |z| of each stream against the *training* standardiser:

clip	lh	rh	pose	face
XXXXXXXXXXXXXXXXXXXXXXXXXXXX	0.65	0.57	1.00	1.34
XXXXXXXXXXXXXXXXXXXXXXXXXXXX	0.83	0.62	0.91	1.58
corpus test clip (reference)	0.91	0.82	1.28	0.89

The videos are **as close to the training distribution as a genuine corpus clip is** on the hand and pose streams. Only the face stream is further out (1.34/1.58 vs 0.89), consistent with a different camera framing (shoulder width 0.21 vs 0.28 — the signer stands further back).

Conclusion: the video → landmark → feature route is sound. The rejection is a model verdict,
not a pipeline bug. That is what made it worth building the decoder rather than debugging the
extractor.

3 • The optimisation: a continuous-signing decoder

New module `modules/continuous.py`:

```
video -> landmarks -> motion energy -> segmentation -> per-segment
recognition
    -> test-time augmentation -> open-set gate -> gloss sequence
```

- **Motion energy** — combined wrist speed in body units (neck origin, shoulder-width scale).
An undetected hand contributes zero motion rather than a jump to the origin.
- **Segmentation** — active runs above an energy floor, short within-sign dips bridged, over-long runs split recursively at their lowest-energy point with a centre bias.
- **Test-time augmentation** — 3 jittered crops per segment, probabilities averaged.
- **Open-set gate** — the v1 fused score and the v1 validation-calibrated threshold, unchanged.

A second, boundary-free decoder (`decode_sliding` : dense sliding window + CTC-style run collapse) was implemented as a comparison.

3.1 How it was tuned — and why the first control set was wrong

The `data_test` videos carry no gloss annotation, so they cannot score anything. Held-out clips can: stitching them into an utterance produces a continuous stream whose gloss sequence is known exactly (`CT.stitch_utterance`).

The first control set inserted a 14-frame rest between signs. That made segmentation far too easy — boundaries were visible as zero-energy valleys — and tuned the decoder to the wrong operating point. **Fluent signing has no rest between signs**: coarticulation means the energy curve never returns to the floor. Confirmed on the real videos, where the entire 271-frame active region is a *single* run with no internal pause.

So a second regime was added and both were used:

- **pause-separated** — `rest=14, blend=6`
- **coarticulated** — `rest=0, blend=10` — what fluent signing and these recordings look like

Settings were selected on **validation**-derived utterances by lowest mean WER across both regimes, then frozen before touching test utterances.

candidate	val paused WER	val fluent WER	mean
A pause-optimal (gap 18, pct 12, min 24, max 100)	0.449	0.814	0.631
B fluent-optimal (gap 18, pct 25, min 30, max 90)	0.576	0.754	0.665
C balanced-1 (gap 18, pct 20, min 28, max 95)	0.525	0.771	0.648

D balanced-2 (gap 15, pct 15, min 28, max 95)	0.525	0.780	0.653
E — selected (gap 18, pct 18, min 26, max 100)	0.466	0.771	0.619

Two bugs were found and fixed along the way:

1. **Over-segmentation.** The first version (`active_pct=35` , `gap=6`) split *within* signs and produced 211 segments for 134 true signs — **WER 1.045**, worse than emitting nothing.
2. **Unbalanced splitting.** The greedy left-to-right split took the global energy minimum and left a long unbalanced tail (a 129-frame segment survived a `max_len=100` cap). Replaced with a centre-biased recursive split, which is what actually finds boundaries in fluent signing.

3.2 Decoder results against ground truth (held-out test utterances)

regime	true signs	decoded segments	WER	CER	exact-match
pause-separated	152	166	0.3618	0.2648	0.250
coarticulated	152	161	0.6447	0.5049	0.025

Comparison points:

decoder	test WER
first version (pct 35 / gap 6)	1.045
sliding-window + run collapse	0.671
segmentation decoder (shipped)	0.362 (paused) / 0.645 (coarticulated)

Segment counts land within ~10 % of the true number of signs in **both** regimes — the segmenter

finds roughly the right boundaries. Recognition holds up when signs are pause-separated and

degrades sharply under coarticulation. **That gap is the model, not the segmenter:** an isolated-sign classifier is sensitive to exactly where the boundary falls, and it has never seen a sign whose start is contaminated by the previous sign's ending.

4 • Result on the two real videos

Shipped config, applied to the actual recordings:

.mp4 — 4 segments decoded (the clause has ~4 signs)

seg	frames	len	output	best guess (below gate)	p	open-set	proto cos
0	42–81	39		0923	0.063	−2.76	0.336
1	75–167	92		0269	0.240	−2.02	0.469
2	161–223	62		0140	0.056	−2.46	0.411
3	217–308	91		0934	0.182	−2.30	0.434

.mp4 — 3 segments decoded

seg	frames	len	output	best guess (below gate)	p	open-set	proto cos
0	0-51	51		0273	0.055	-2.82	0.306
1	45-144	99		0150	0.065	-2.47	0.413
2	138-168	30		0920	0.054	-2.78	0.326

Output sequences: and .

What improved: the pipeline now finds a plausible number of signs and localises each one in

time, instead of returning a single label for a 10-second sentence. The notebook plots the motion

energy with the decoded segment boundaries and a mid-stroke skeleton per segment, so the segmentation is inspectable.

What did not: every segment is still rejected. Prototype similarities are **0.31–0.47**

against 0.767 ± 0.171 for in-vocabulary material — roughly 2 standard deviations below.

There is no threshold that accepts these without also accepting noise; lowering the gate to admit

them would admit essentially anything.

Given a **184-gloss isolated-sign vocabulary** and two free-form Thai sentences, rejecting is the correct and safe behaviour. The system says "I saw four signs and I do not know any of them" rather than inventing four Thai words.

5 • Honest limits of this add-on

1. **The rejections cannot be attributed.** Two explanations fit equally well: the signs are genuinely outside the 184-gloss vocabulary, or they are inside it but unrecognisable under coarticulation (§ 3.2 shows WER rises from 0.36 to 0.64 under coarticulation alone). Separating them needs either the Thai lemma table (to check vocabulary coverage) or a gloss-level annotation of these two videos. **Neither exists in the workspace.**
 2. **Nothing here is scored against the real videos.** They have no ground truth. Every number in § 3.2 comes from stitched held-out clips; the real videos are diagnosed, not measured.
 3. **Stitched utterances are a proxy for continuous signing, not the real thing.** They reproduce coarticulated boundaries but not co-articulated *handshapes*, prosody, or the grammatical inflections real signers use in sentences.
 4. **Two videos is not a sample.** These findings describe two recordings.
 5. **No retraining was done.** The add-on is a decoder around the frozen v1 checkpoint.
-

6 • What would actually make these videos produce Thai text

In order of expected payoff:

1. **Gloss-level annotation of a handful of continuous videos.** Even 20 annotated utterances would resolve § 5.1 and give a true continuous-signing WER instead of a proxy.
2. **Train on continuous data.** Fine-tune the trunk on stitched coarticulated utterances with a

CTC head over the existing 256-D embedding. This directly attacks the 0.36 → 0.64 gap and needs no new recordings — the stitching code is already in `modules/continuous.py`.

3. **The Thai lemma table.** Turns gloss ids into words, activates the LLM and TTS stages for real, and makes vocabulary coverage checkable.
4. **Vocabulary expansion.** 184 glosses will not cover free-form conversation; the sentences in `data_test/` use everyday vocabulary that a 184-word lexicon is unlikely to contain.
5. **Re-extract with Face Mesh** (carried over from v1) — the face stream is also the most out-of-distribution channel on these videos ($|z|$ 1.34/1.58), so better facial features would help both accuracy and domain robustness.

7 • Changes to the workspace

path	change
<code>modules/continuous.py</code>	new — motion energy, segmentation, TTA decode, sliding-window decode, utterance stitching, extraction sanity, distribution shift
<code>slm_labs.ipynb</code> § 15.2	rewritten — extraction + convention check + distribution shift + naive route + continuous decode + energy/segment plots + per-segment skeletons + ground-truth control
<code>slm_labs.ipynb</code> § 16	continuous-decoding and <code>data_test</code> rows added to the consolidated table
<code>prompt/result/blr_v1_2.md</code>	this document

No model was retrained; `artifacts/mssf_best.pt` is unchanged. No other file was touched.